



INFORMÁTICA II

Bioingeniería



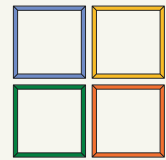
Índice



**Función lambda
y funciones**



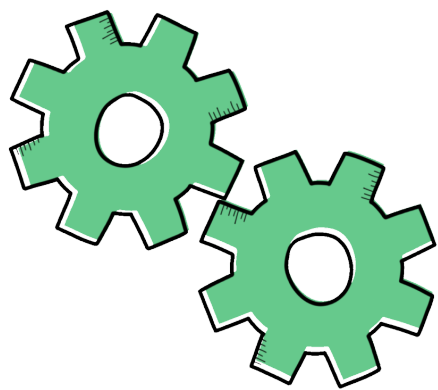
Manipulación

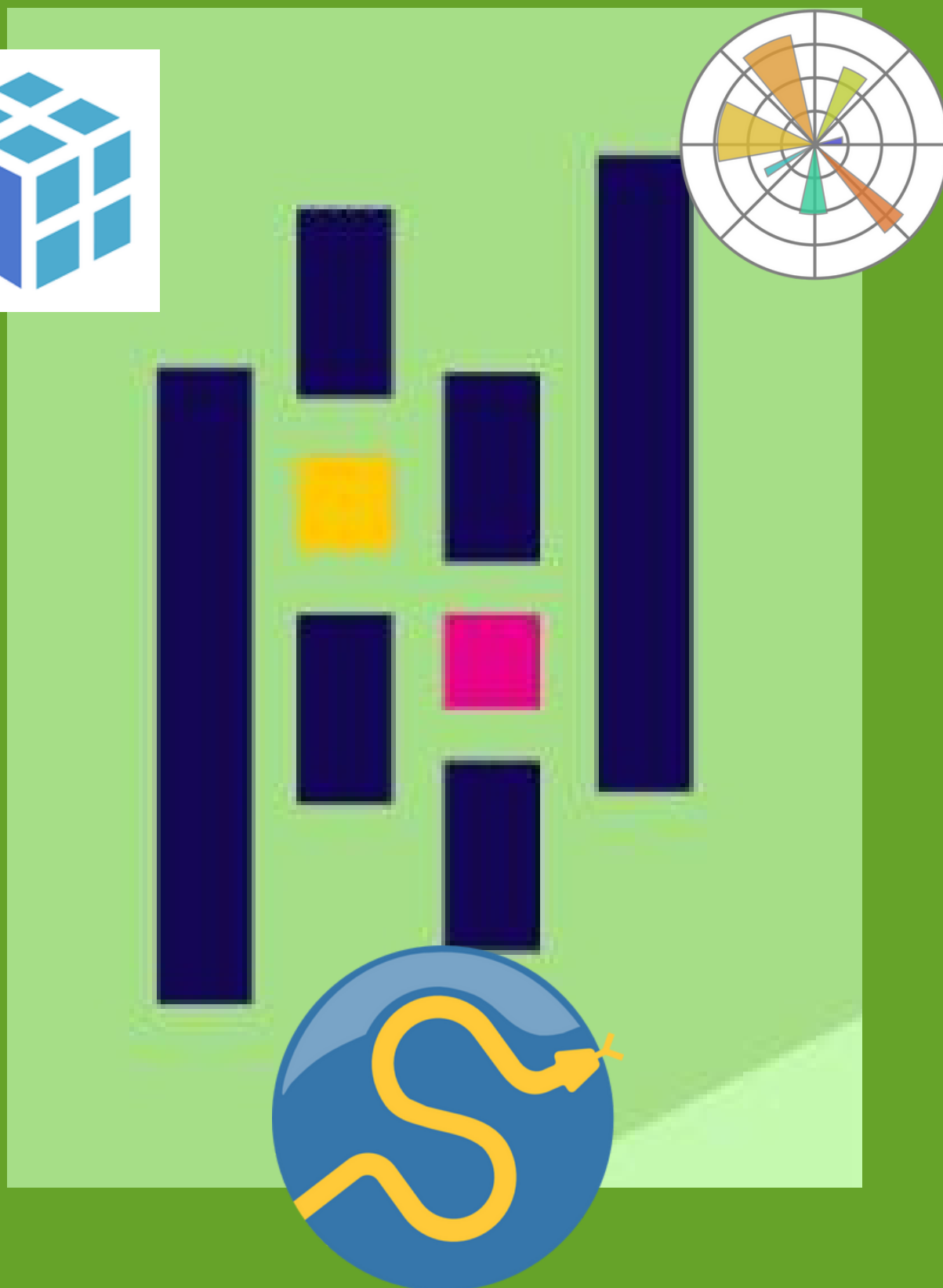


Graficación



Ejercicios



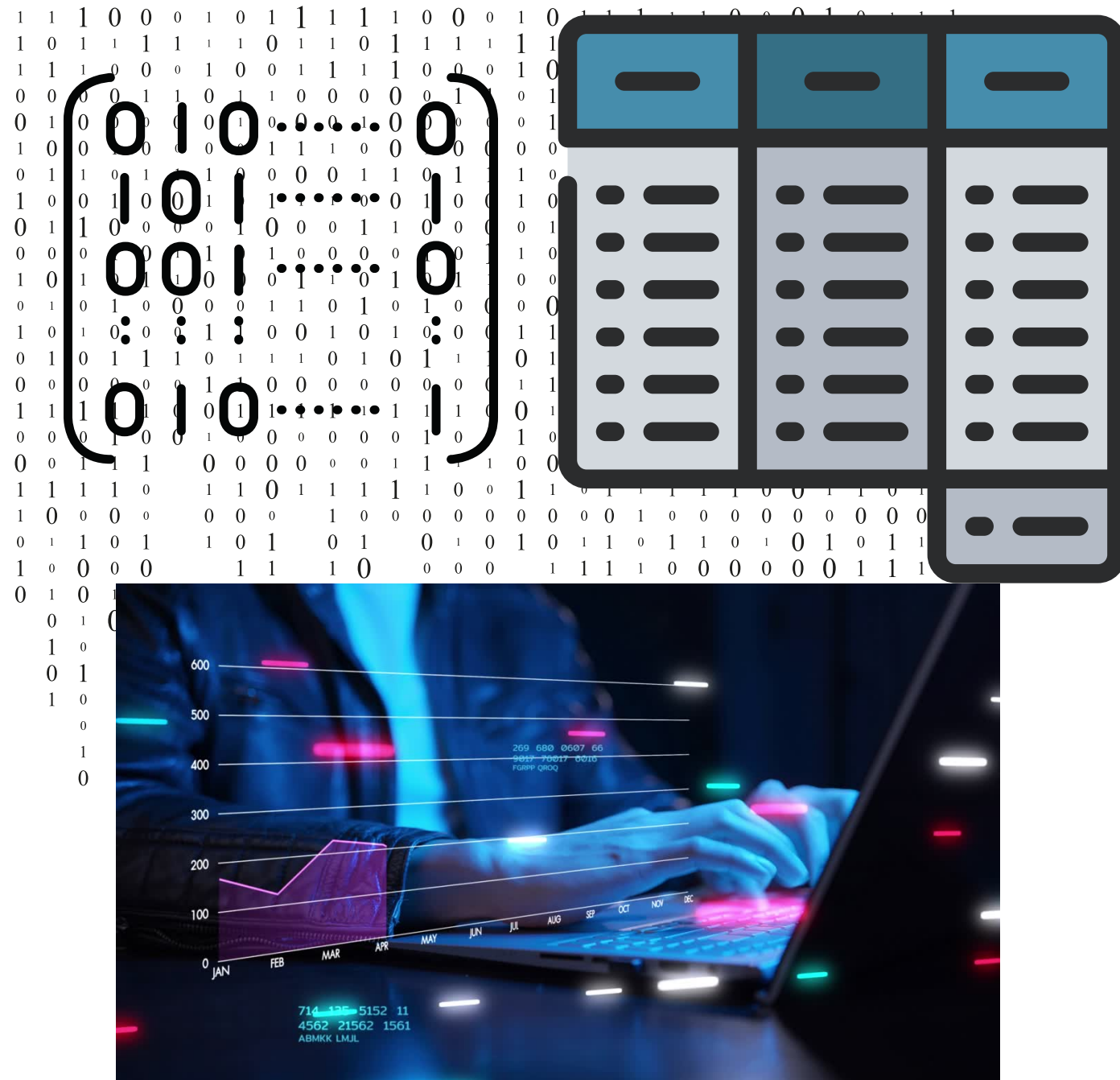


Unidad 2

Pandas



Función lambda y funciones



Al implementar nuestras propias funciones, se debe tener presente que se hará sobre todo el dataframe (la matriz) o sobre una serie (columna), para este fin se usa el metodo **apply, applymap o map** de la siguiente forma.

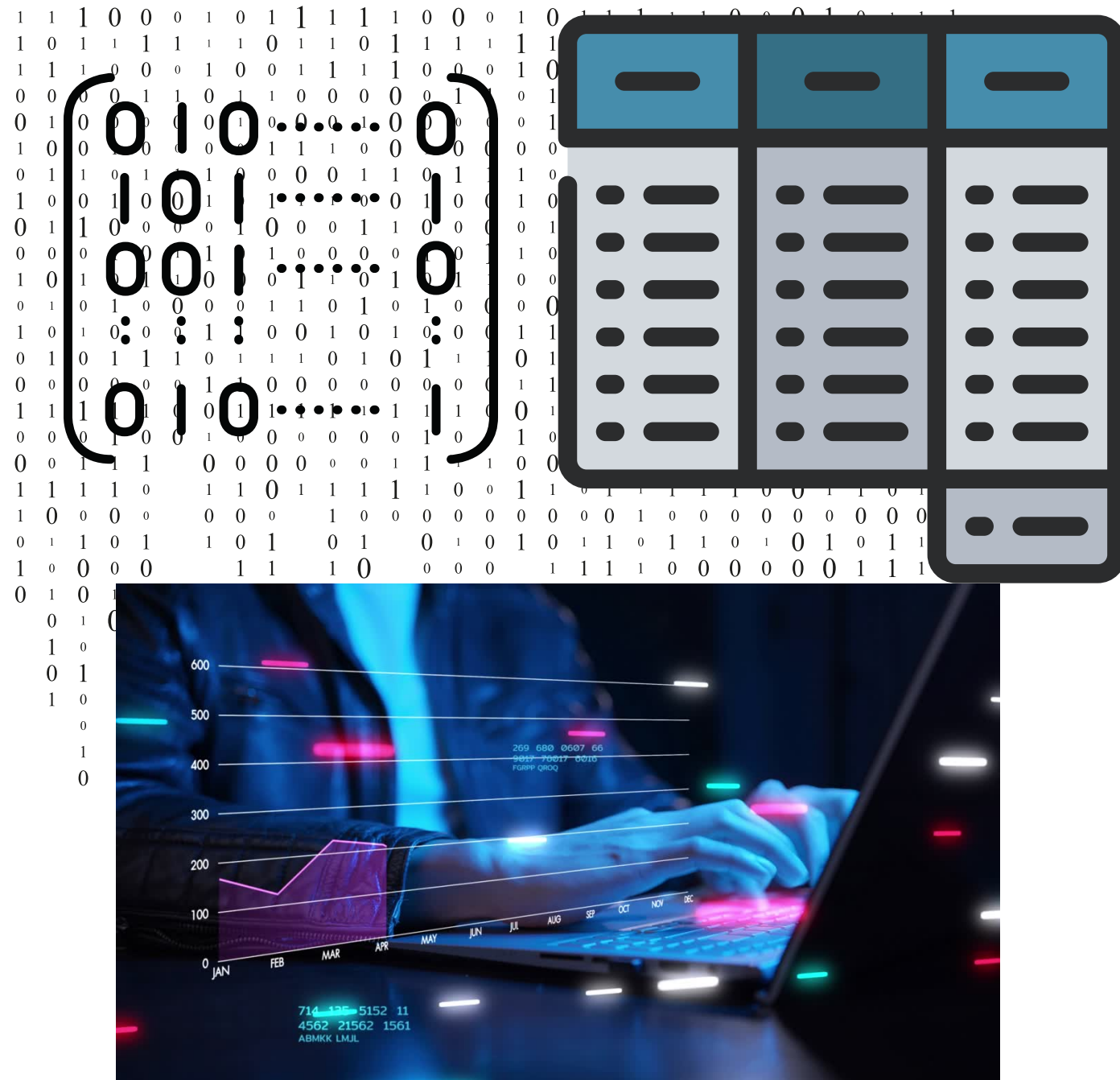
```
def fun_1(x):  
    y = x**2 + 1  
    return y
```

`df.apply(fun_1)` # Se aplica sobre un eje a todo el dataframe

`df['hour'].apply(fun_1)` # Se aplica a una serie



Función lambda y funciones



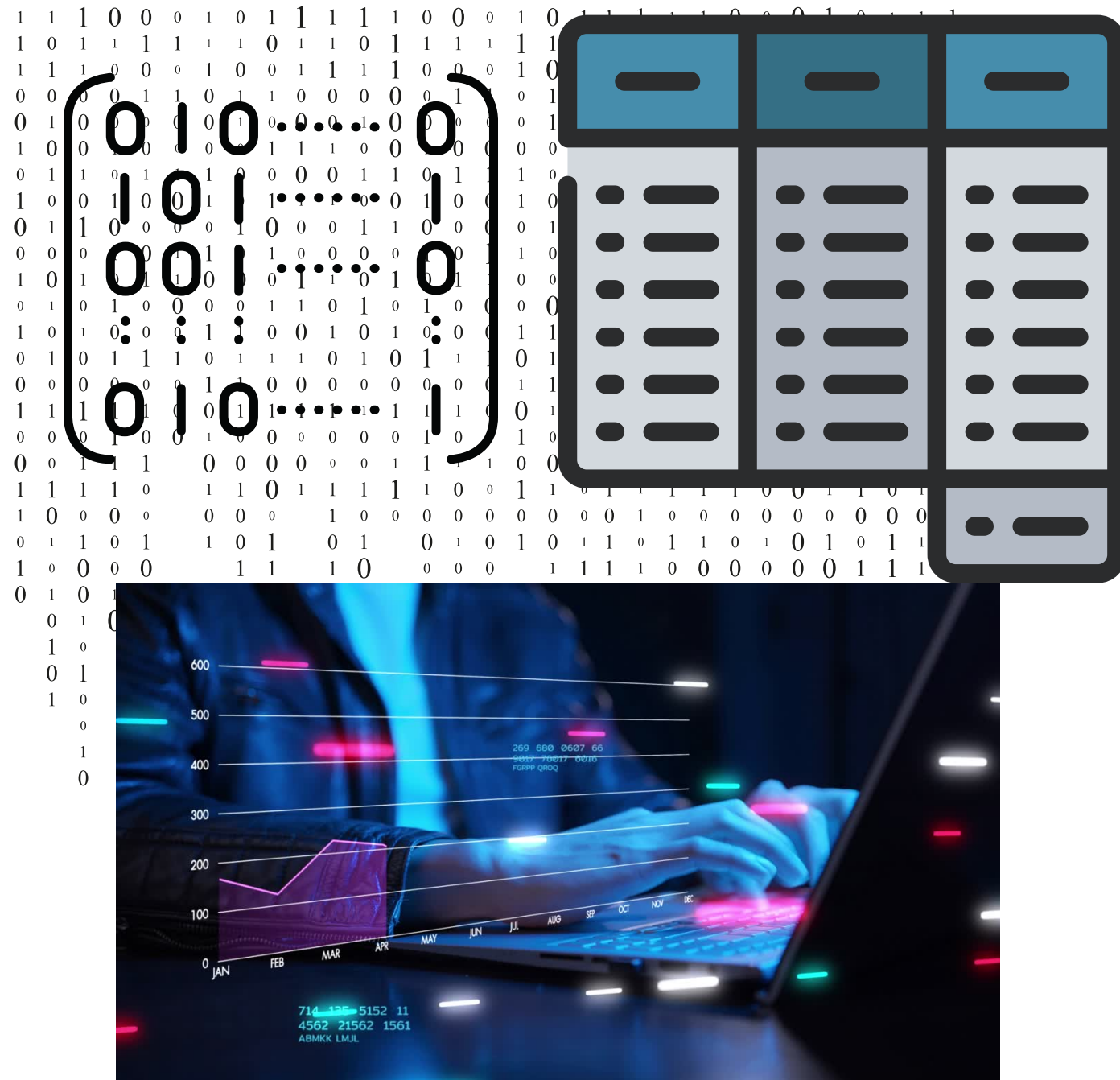
Otras formas de implementación:

```
def fun_2(x, a =1, b = 0):  
    y = x**2 + a*x + b  
    return y
```

```
df['hour'].apply(fun_2, args = (20, -100))  
df['hour'].apply(fun_2, a =20, b= -100)
```



Función lambda y funciones

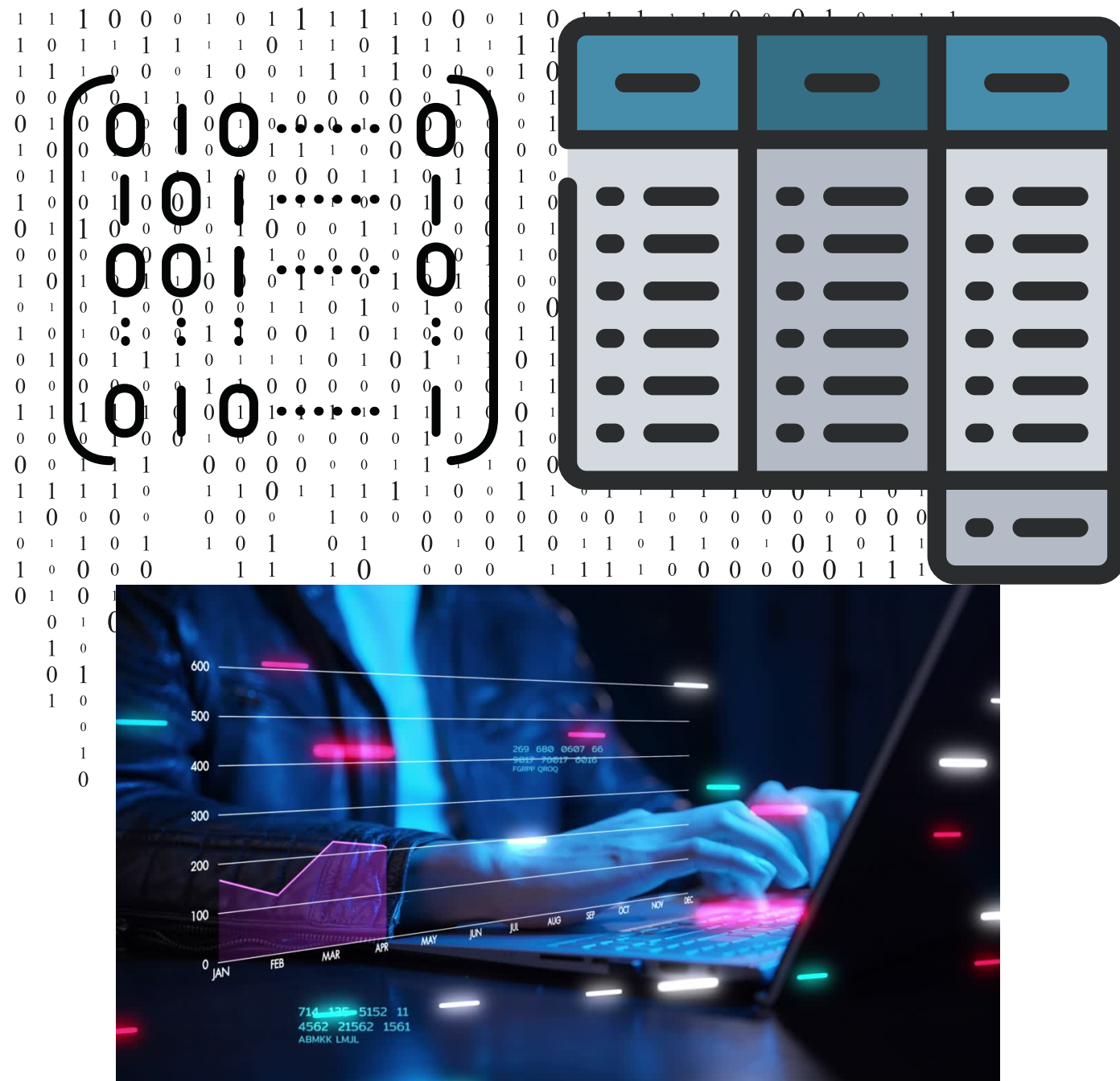


Implementandolo con ***Lambda*** sobre un eje

`df['t1'].apply(lambda x: x+273)` # Sobre una serie
`df.apply(lambda x: x.mean())` # Sobre el dataframe
a lo largo del eje 0 por defecto
`df.apply(lambda x: x.mean(), axis=1)` # sobre eje 1



Función lambda y funciones



Implementandolo con ***Lambda*** sobre un eje

```
df.applymap(lambda x: x/1000)
```

Se aplica sobre todo el dataframe

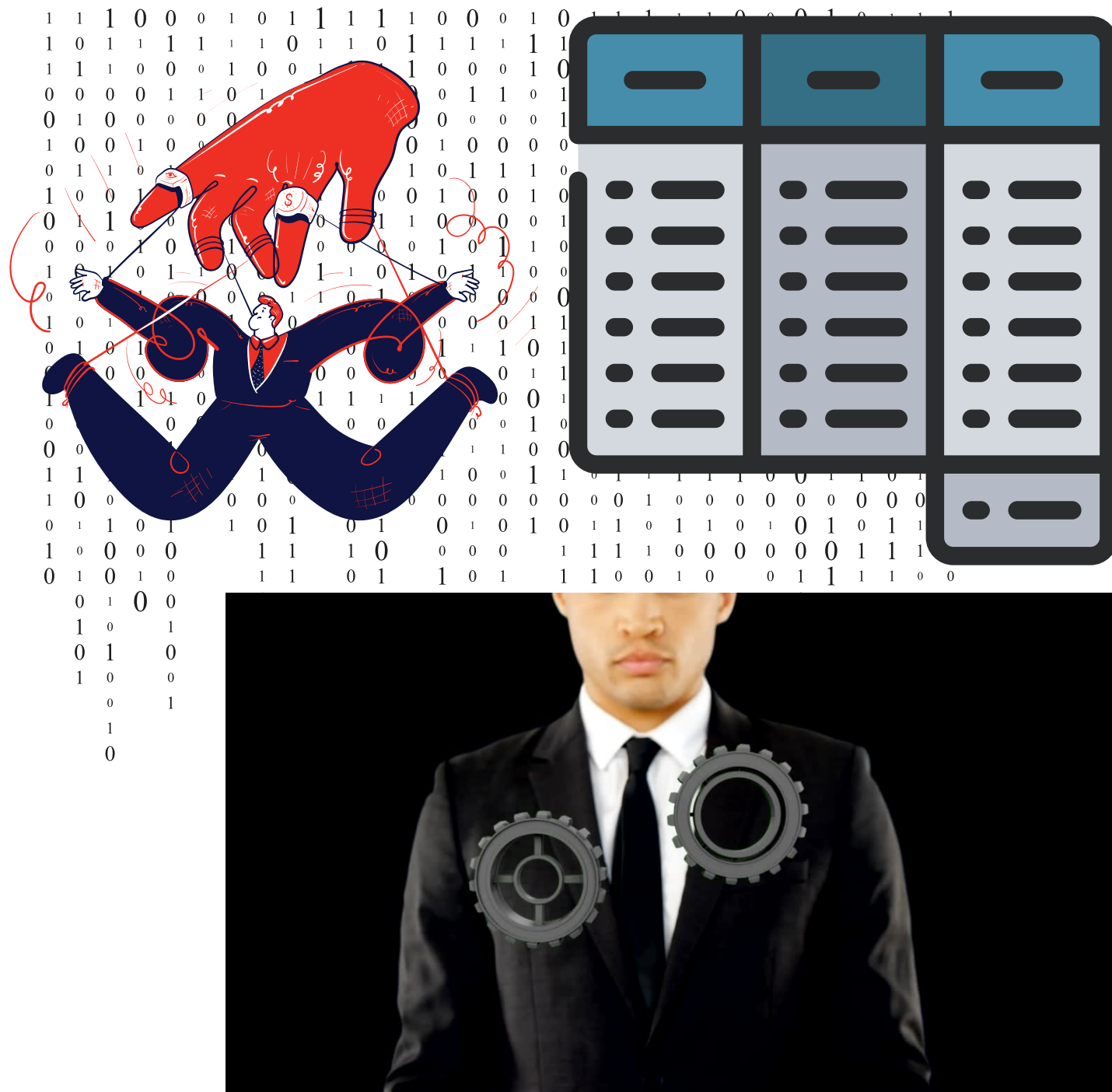
```
df['hour'].applymap(lambda x: x/1000) # Se aplica sobre una serie
```

Nota: Los métodos map y apply se aplican sobre series con la diferencia que apply permite hacerlo a lo largo de un eje , y map sobre columnas (series) nada mas.

El método applymap se aplica sobre cada elemento que compone el DataFrame



Manipulación de datos

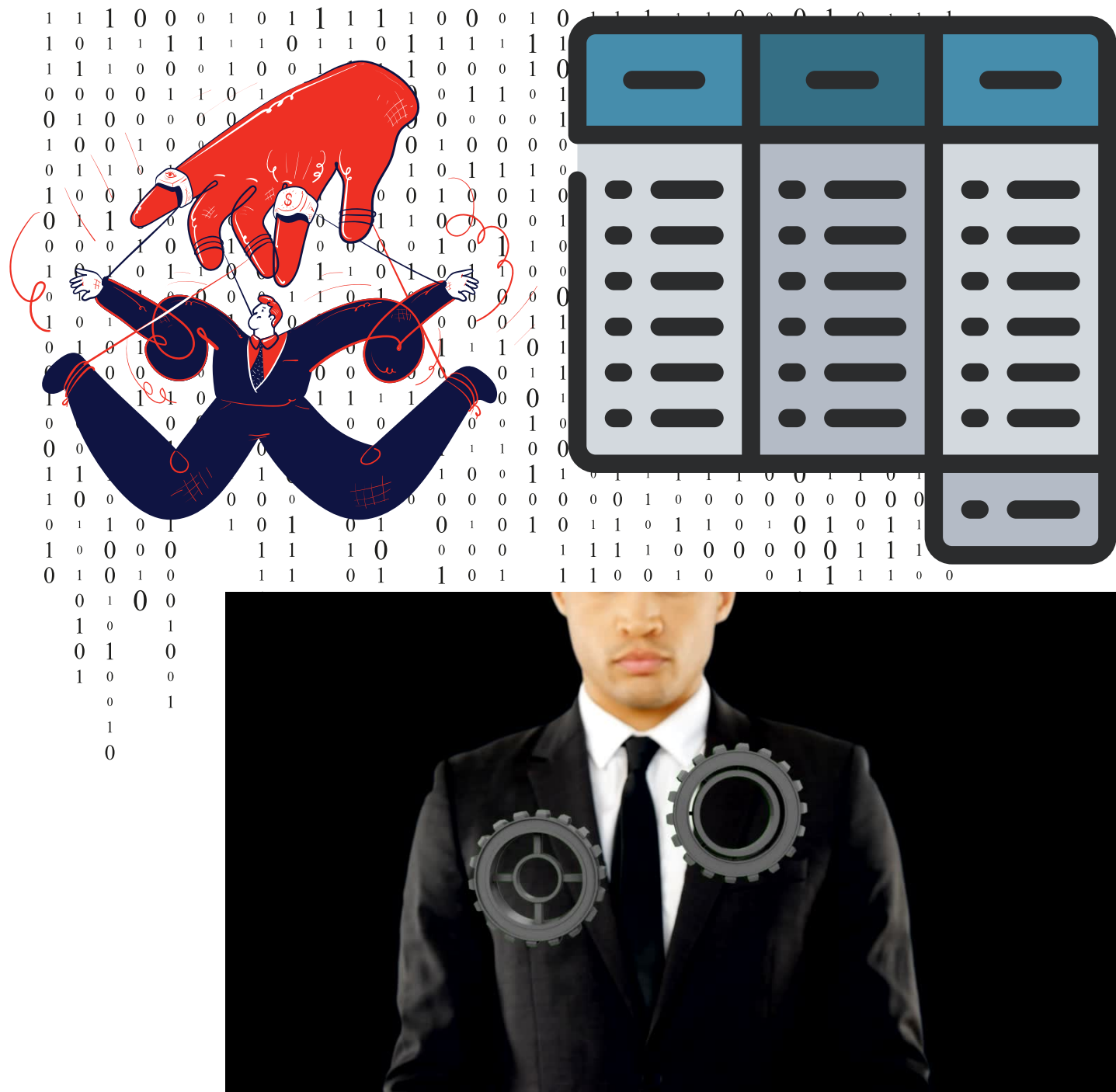


La gran ventaja de Pandas, en la manipulación que este permite de la información, da pie a crear nuevas columnas, modificarlas, borrarlas, agruparlas, etc..., a partir de otras con base al análisis que se haga de los datos.





Manipulación de datos



Crear nuevas columnas

La sintaxis para crear nuevas columnas es exactamente igual que en los diccionarios crear nuevos elementos

```
df['speed'] = df['distancia']/df['time']
```

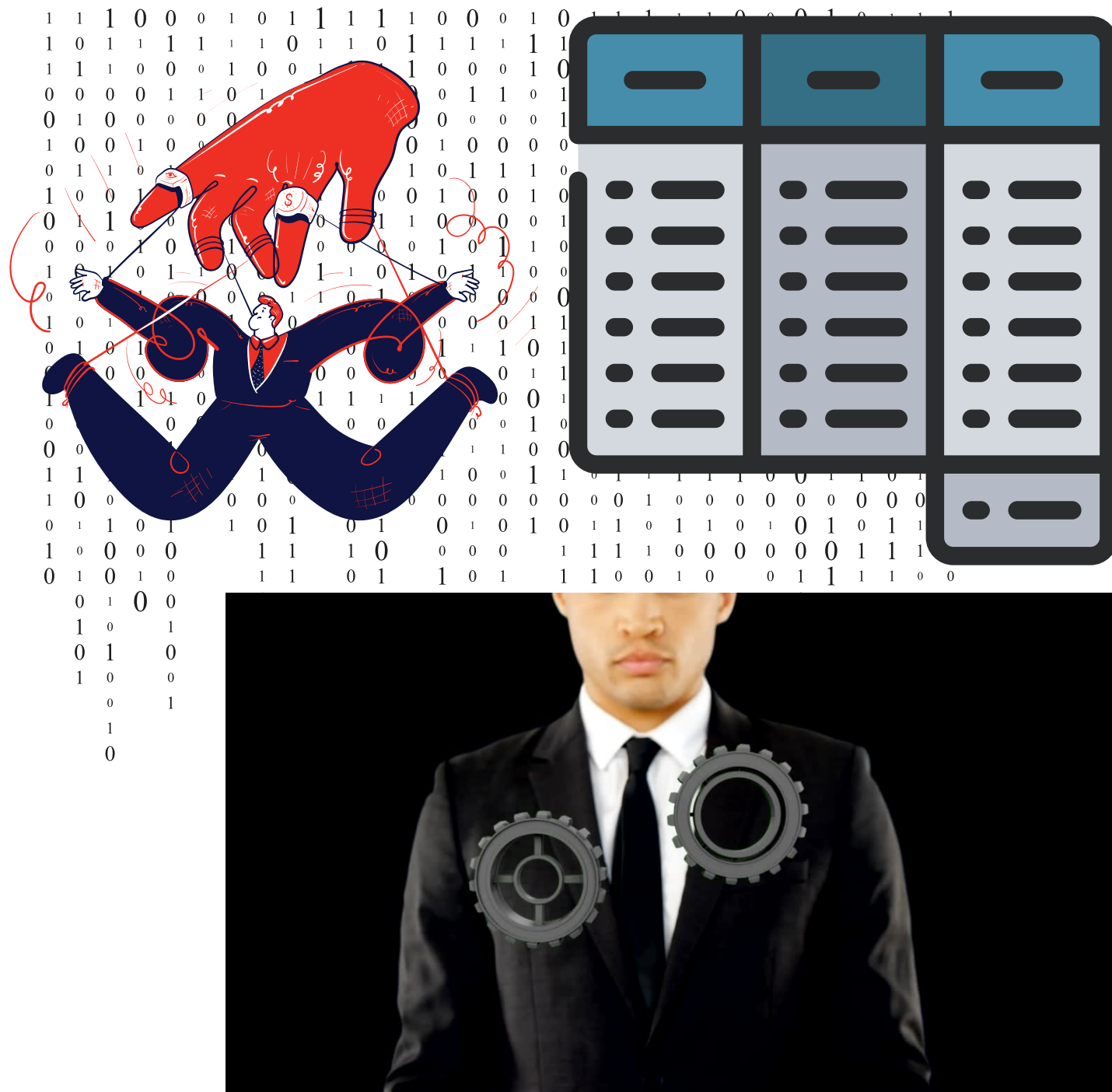
Se pueden crear a partir de series del mismo dataframe o a partir de otros arreglos

```
df['speed'] = "No aplica"
```





Manipulación de datos



Modificar columnas

Se llama la columna que se desea modificar y se le ingresa el valor deseado

```
df.loc[df['genero']=="F",'genero']=1  
df.loc[df['genero']=="M",'genero']=0  
df['genero'] = df['genero'].replace({"M":0,"F":1})
```

Se modificar a partir de series del mismo dataframe o a partir de otros arreglos

```
df.loc[df['Promedio']<6,'Aprobar'] = False
```



Manipulación de datos



Agrupar

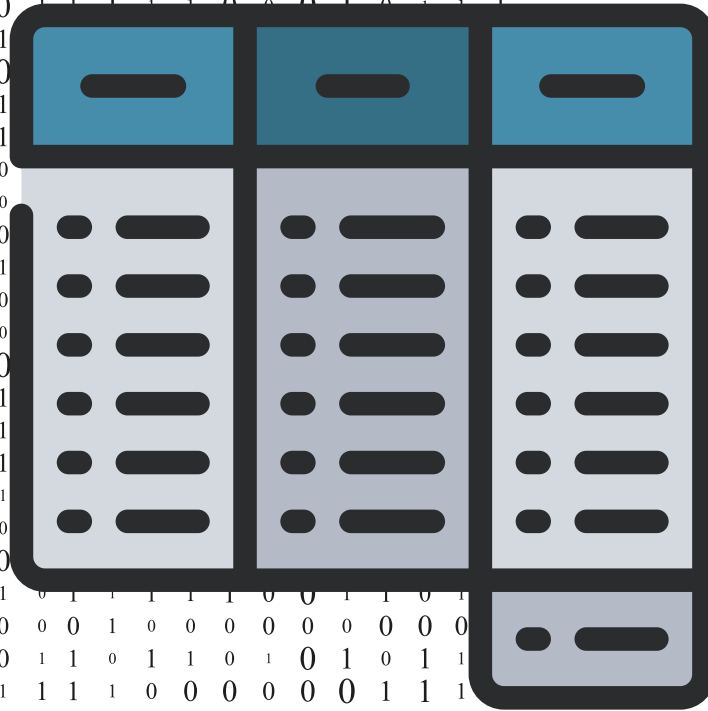
Se usa el metodo ***groupby()*** que creará in objeto tipo grupo donde esta el dataframe reordenado en los grupos que se indicó

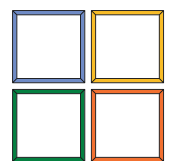
`df.groupby(['genero','pais'])`

Sobre estos podemos agrupamiento podemos indezar o realizar operaciones y después visualizarlas como cualquier dataframe

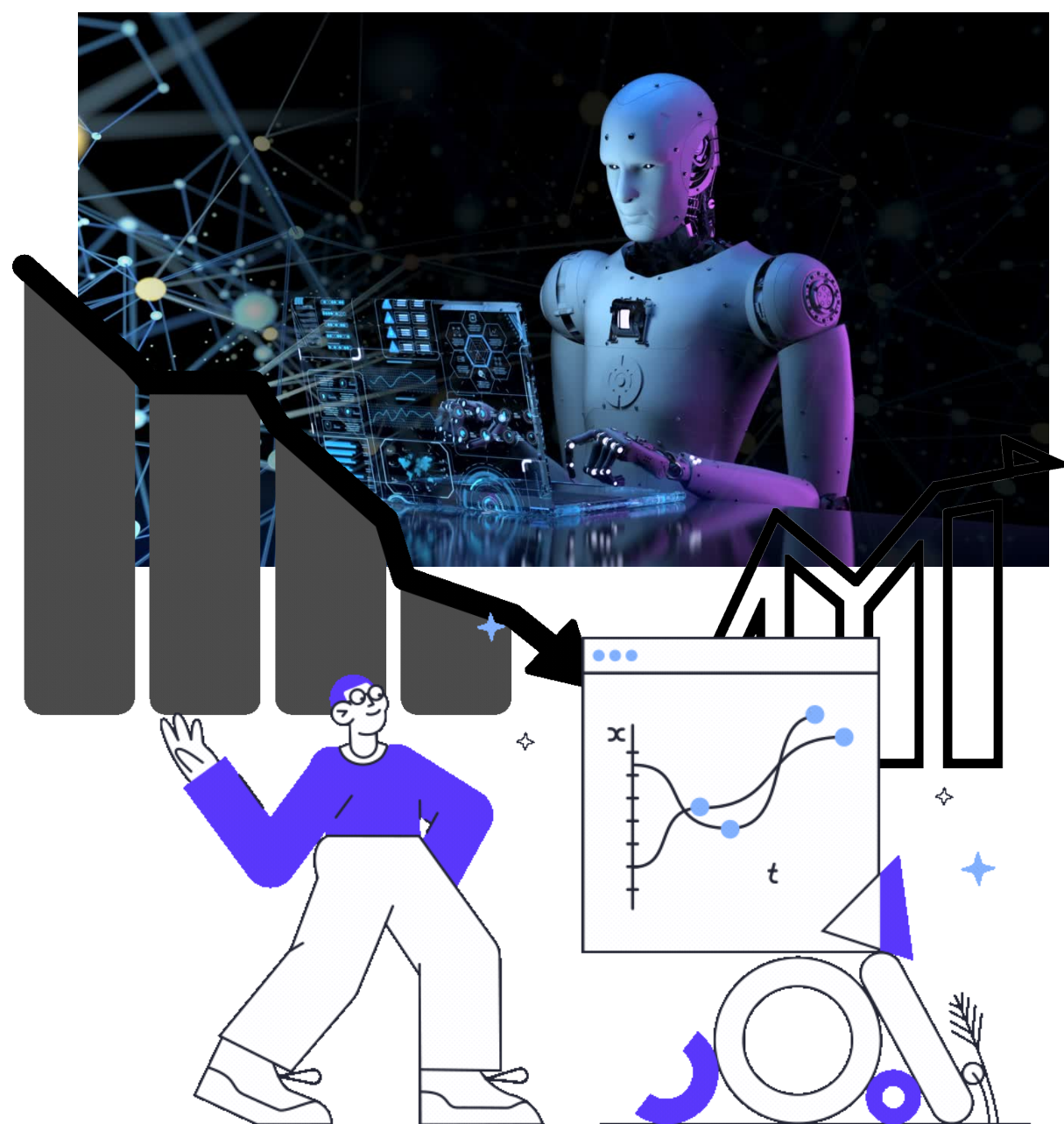
`df.groupby(['genero','pais']).mean()`

`df.groupby(['genero','pais'])["cm"].std()`



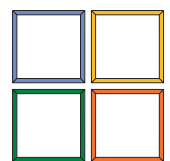


Graficación a partir de objetos de Pandas



Cómo hemos visto Pandas tiene métodos que también son usados en Numpy con ciertas diferencias y optimizaciones, para la graficación veremos que integra muchas funcionalidades de Matplotlib (Polimorfismo)

Nota: Tanto Numpy como matplotlib.pyplot siguen siendo compatibles con los objetos de Pandas , por tanto , es posible hacer operaciones usando Numpy y graficar con Matplotlib, si así lo requieren



Graficación a partir de objetos de Pandas

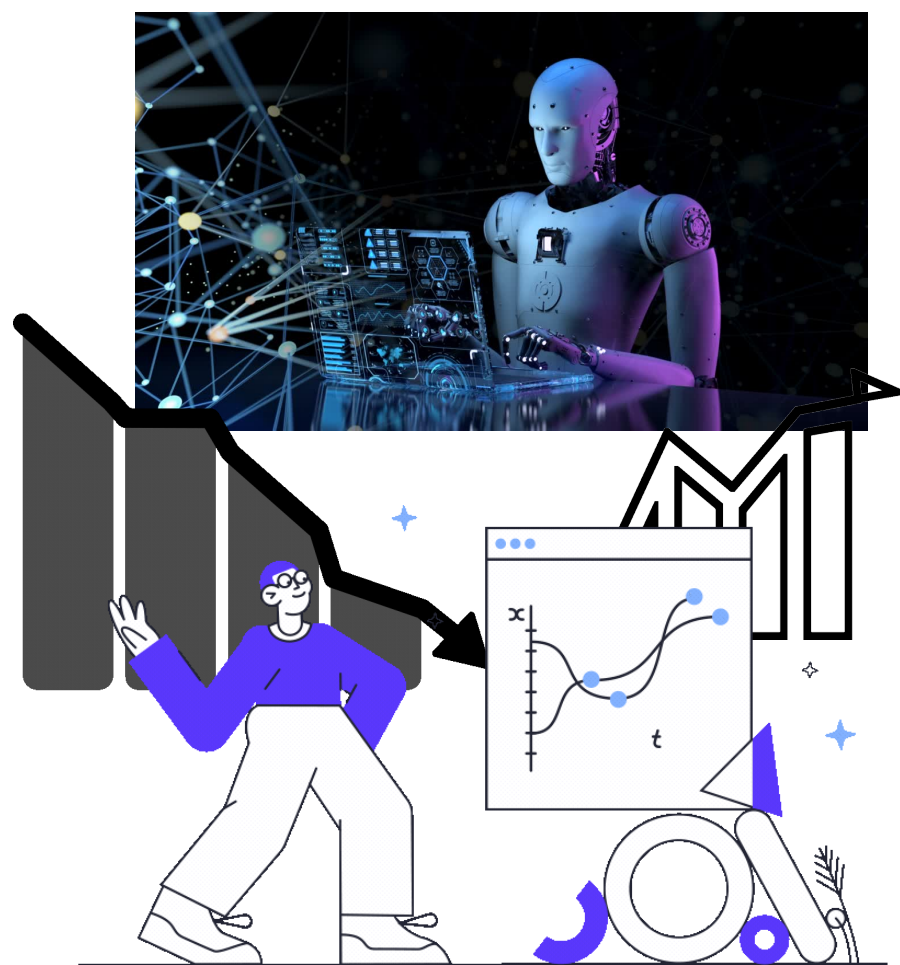


Pandas integra el método ***plot***, y con este y su argumento ***kind*** podemos graficar varios tipos de graficas como las usadas desde Matplotlib.

kind : *str*

The kind of plot to produce:

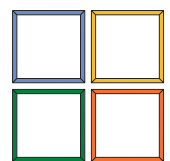
- 'line' : line plot (default)
- 'bar' : vertical bar plot
- 'barh' : horizontal bar plot
- 'hist' : histogram
- 'box' : boxplot
- 'kde' : Kernel Density Estimation plot
- 'density' : same as 'kde'
- 'area' : area plot
- 'pie' : pie plot
- 'scatter' : scatter plot (DataFrame only)
- 'hexbin' : hexbin plot (DataFrame only)



```
df['t1'].plot()  
plt.plot(df['t1'])
```

```
df['t1'].plot.bar()  
df['t1'].plot(kind = 'bar')
```

```
df['t1'].plot.pie()  
df['t1'].plot(kind = 'pie')
```



Graficación a partir de objetos de Pandas

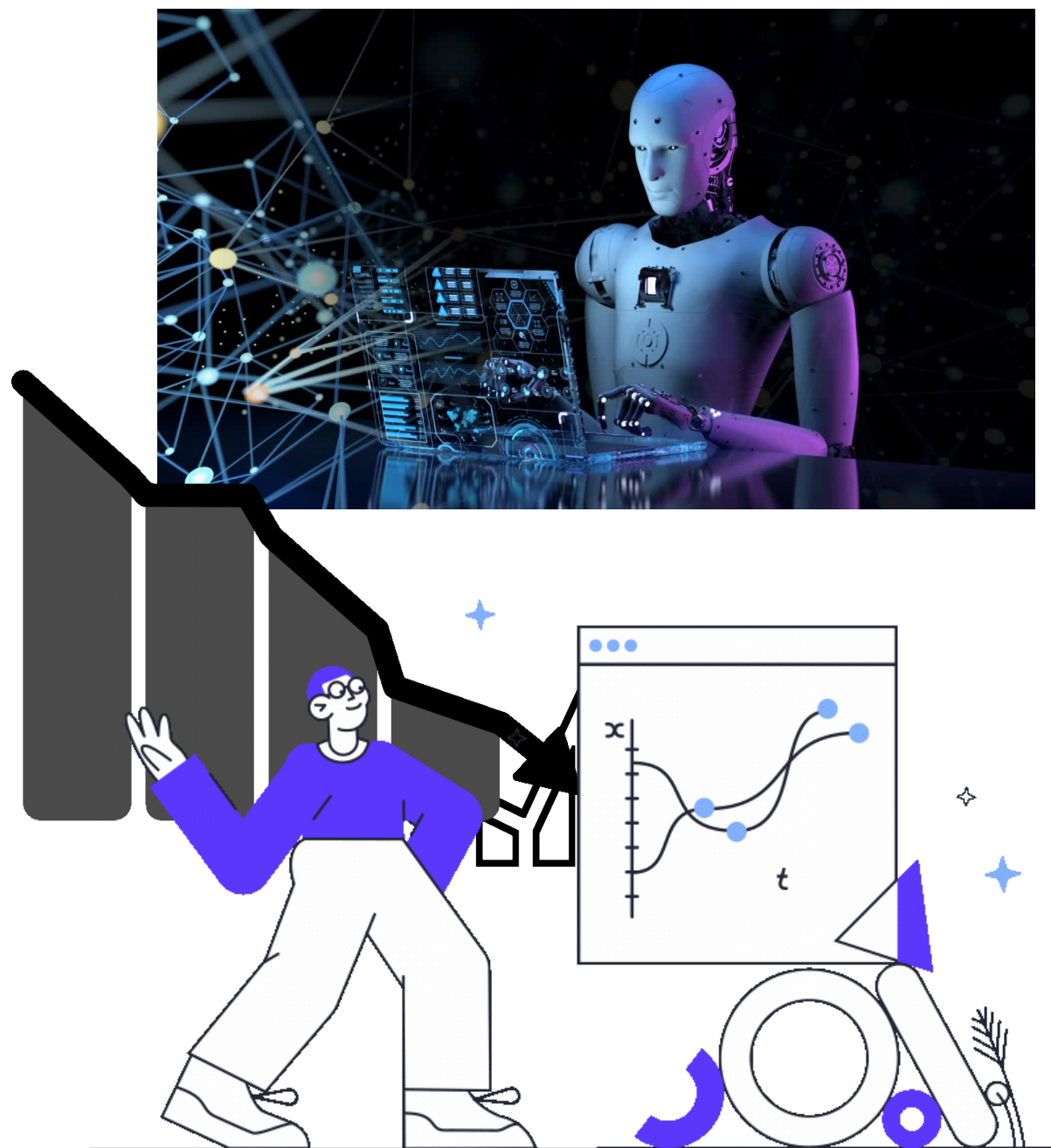


En este caso se parte de la serie que se desea graficar, si el caso es hacerlo con matplotlib, se le pasará como argumento la serie que desea graficar

```
plt.plot(df['t1'])  
plt.bar(df['t1'])  
plt.pie(df['t1'])
```

Pandas tambien integra los método **hist**, **boxplot** de manera directa

```
df['t1'].hist()  
df['t1'].boxplot()
```

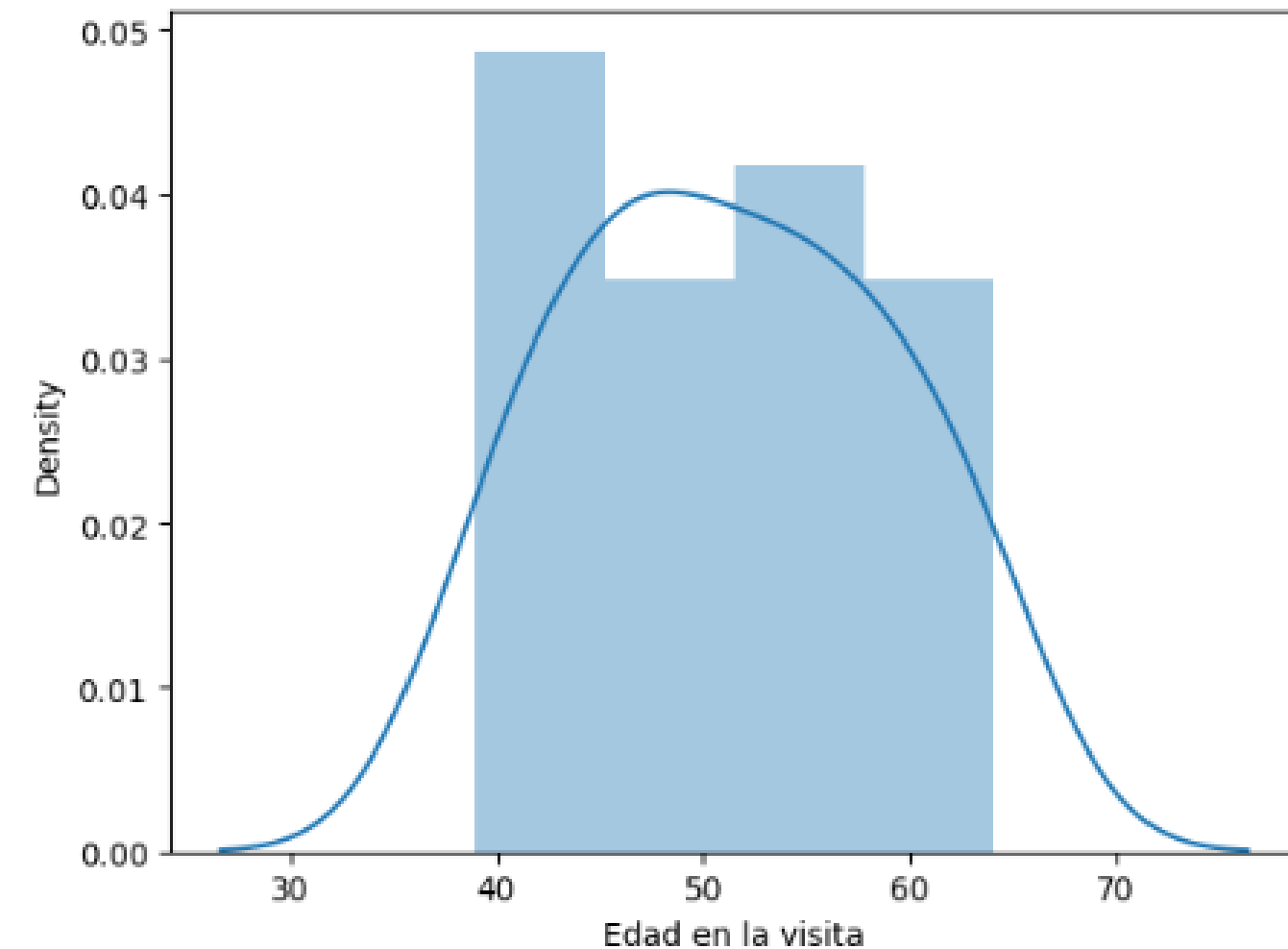
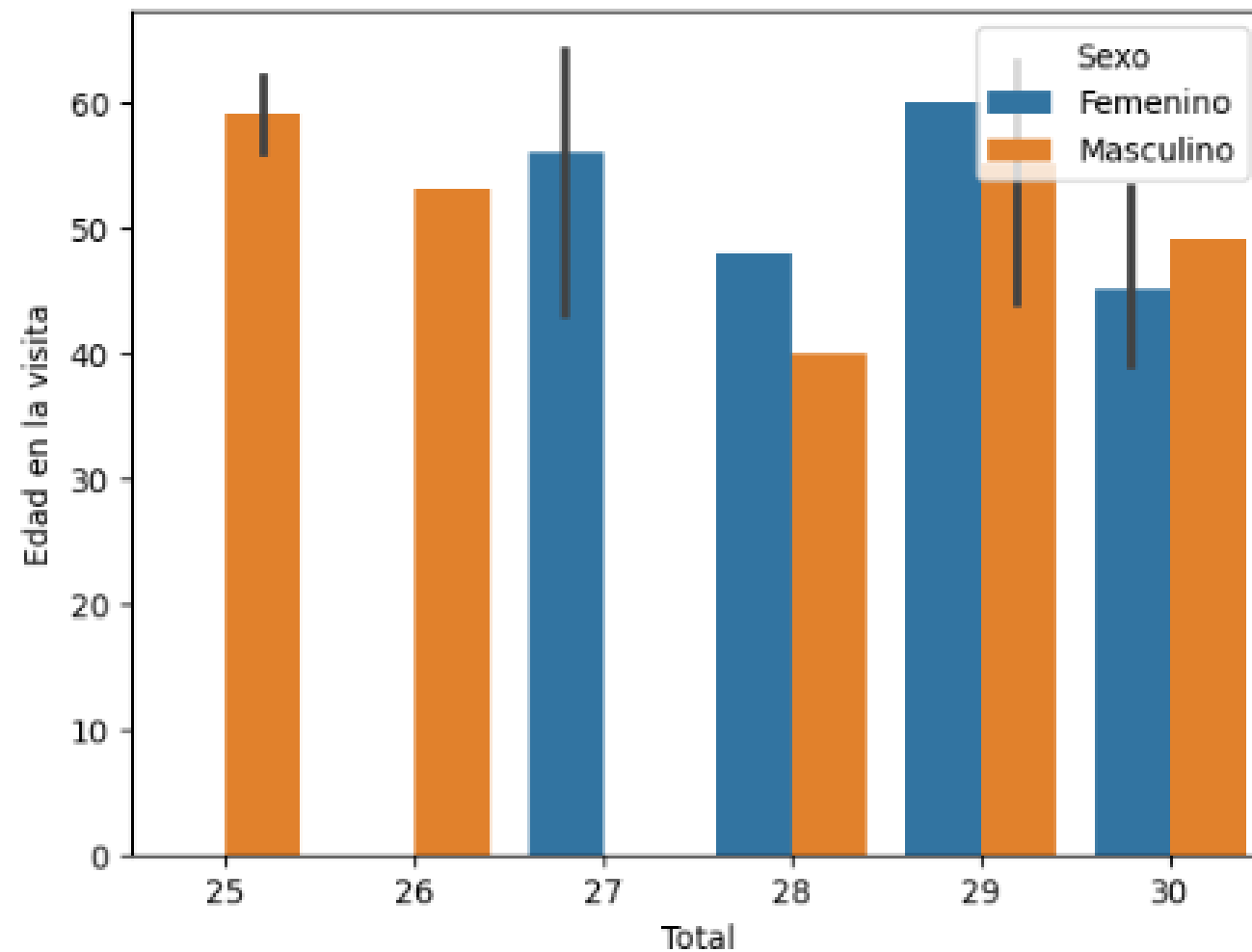


Seaborn

```
import seaborn as sns
```

```
sns.barplot(x='Total', y='Edad en la visita', hue='Sexo', data=mmse, estimator=np.median)
```

```
sns.distplot(mmse['Edad en la visita'])
```

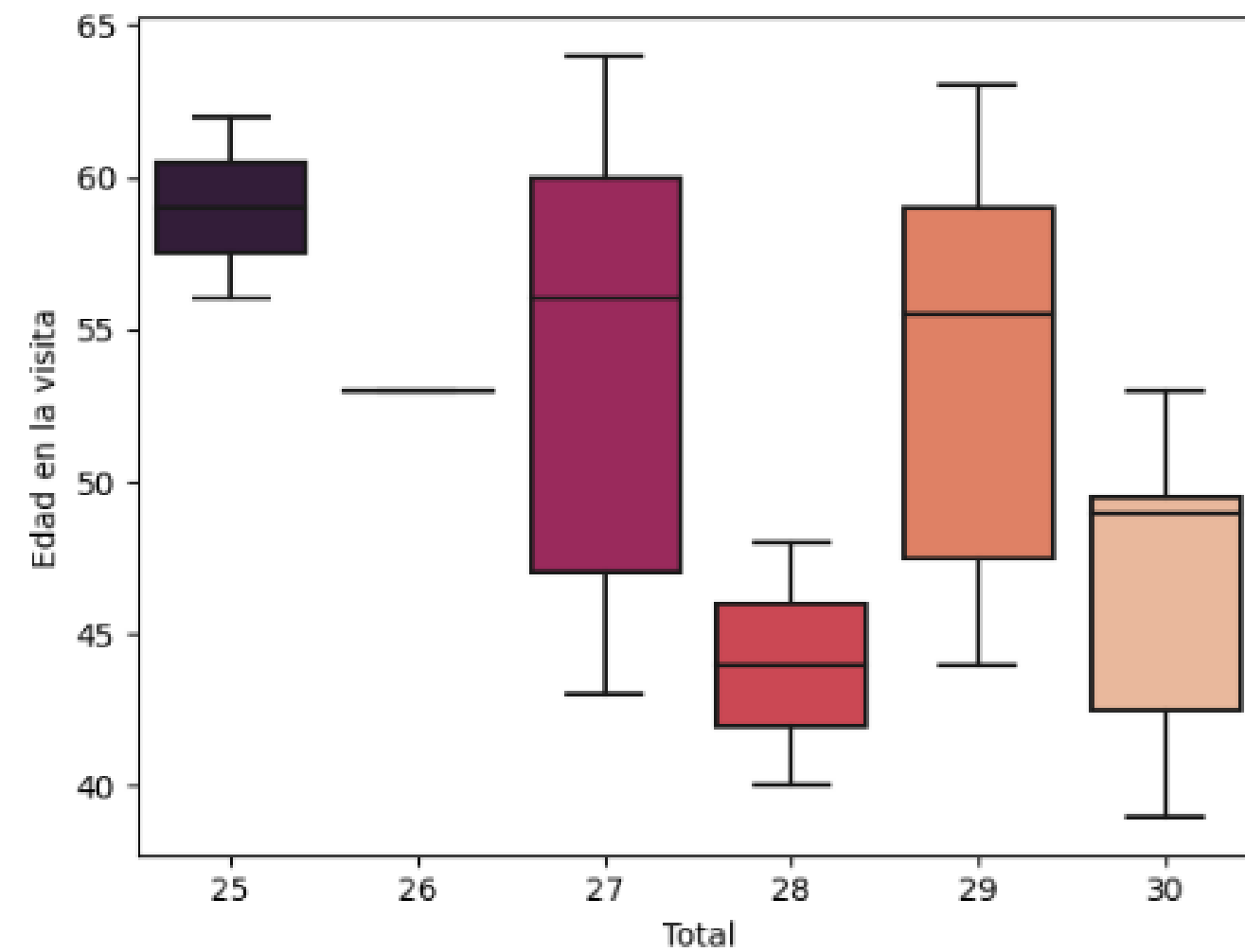
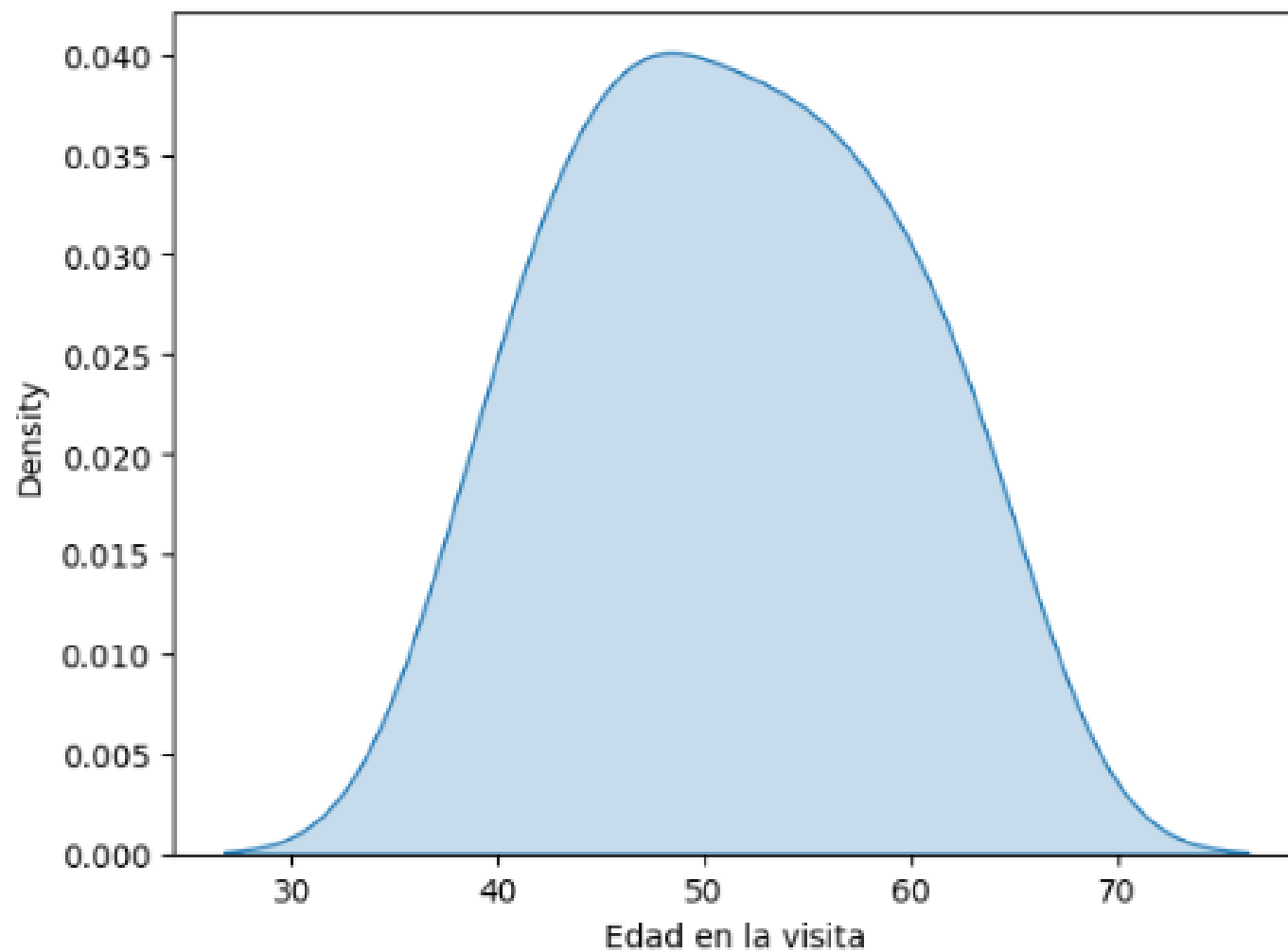


Seaborn

```
import seaborn as sns
```

```
sns.kdeplot(data=mmse['Edad en la visita'], shade=True)
```

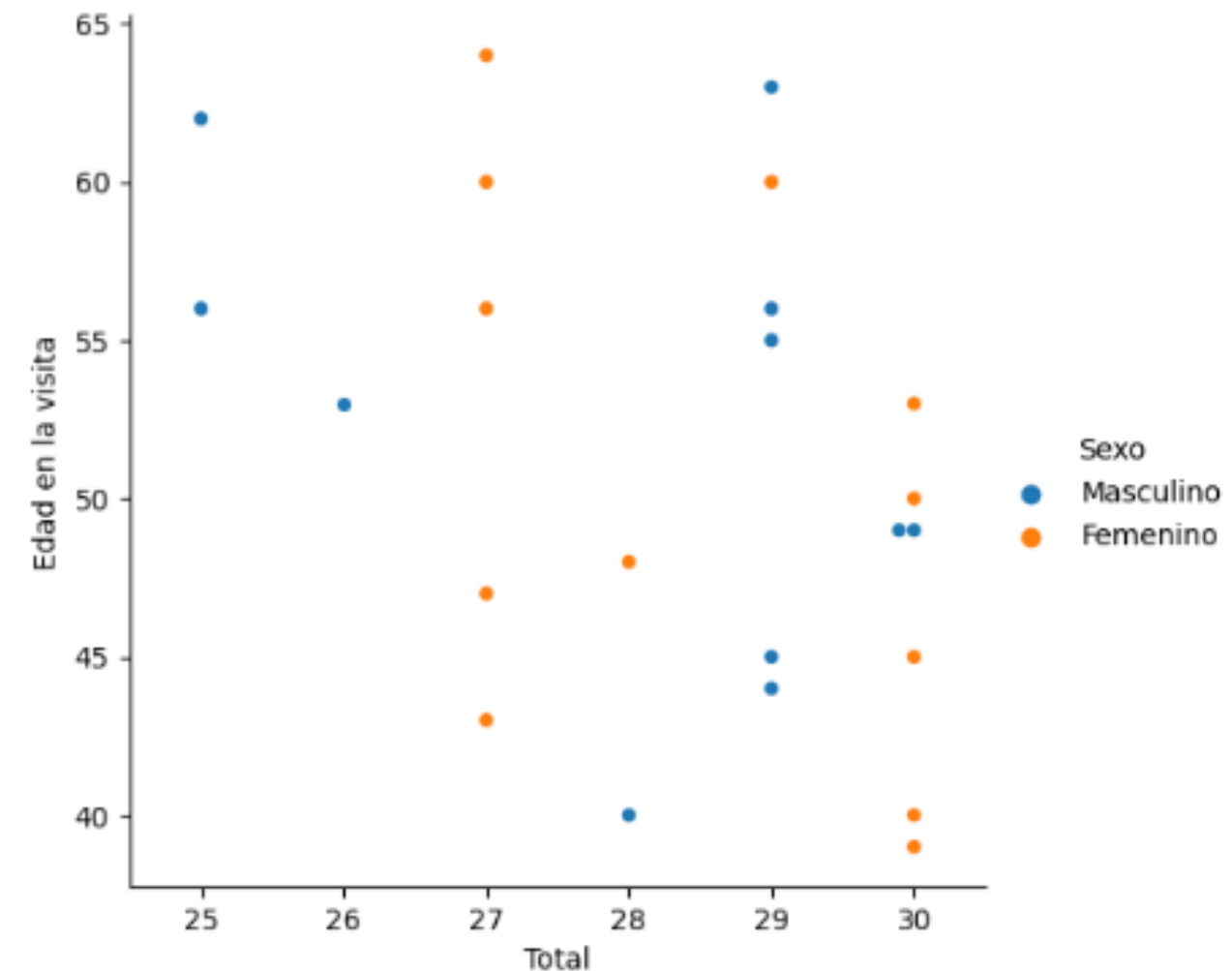
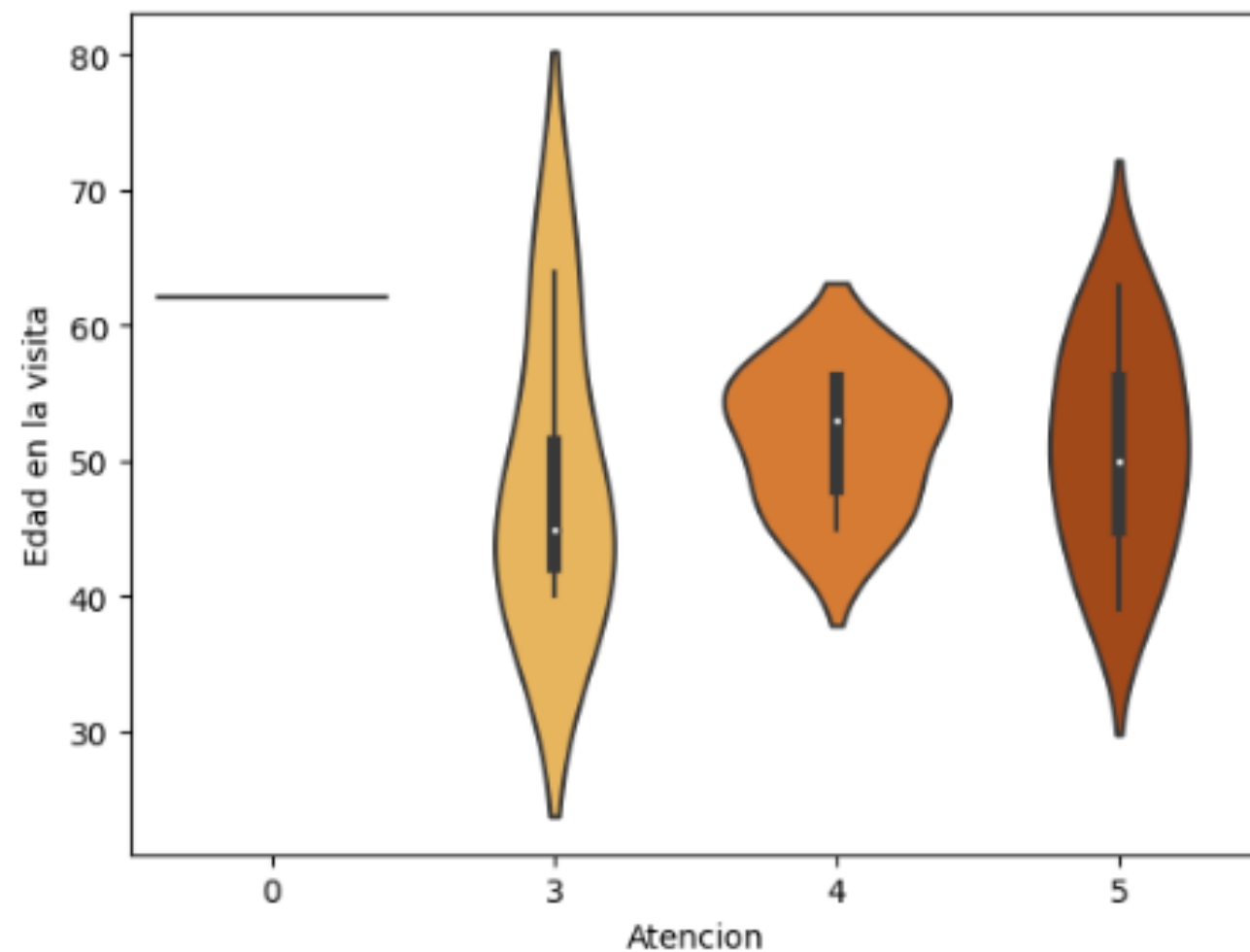
```
sns.boxplot(data=mmse, x='Total', y='Edad en la visita', palette="rocket")
```



Seaborn

```
sns.violinplot(data=mmse, x='Atencion', y='Edad en la visita', palette="YlOrBr")
```

```
sns.catplot(x='Total', y='Edad en la visita', hue='Sexo', kind="swarm", data=mmse)
```



Seaborn

```
penguins = sns.load_dataset("penguins")  
sns.jointplot(data=penguins, x="flipper_length_mm", y="bill_length_mm", hue="species")
```



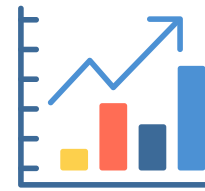


Ejercicios



Crear un sistema que almacene equipos biomédicos , cada equipo tiene información de nombre, marca, ubicación, fecha de calibración ,fecha de mmtto ,nombre del proveedor.

Para ello implementarlo a partir de un dataframe con la información anterior



Ejercicios



De la siguiente pagina:

https://www.kaggle.com/datasets/imdevskp/corona-virus-report?select=covid_19_clean_complete.csv

Extraer el datasets de datos , leer y analizarlos , mirar como pueden ser agrupados y que información podría mostrar según sus columnas, mirar promedio , desviación estándar,etc.



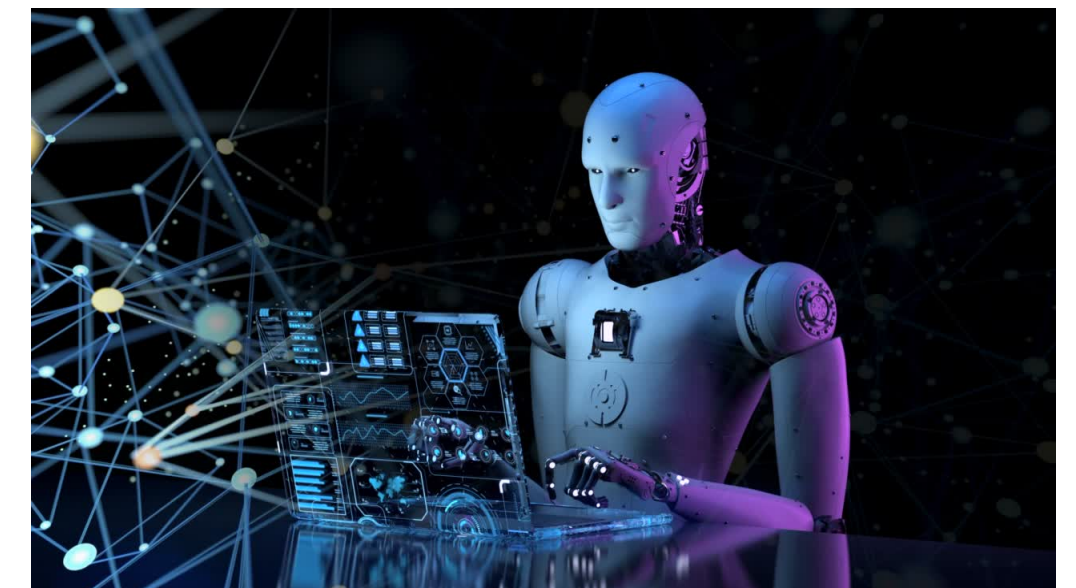
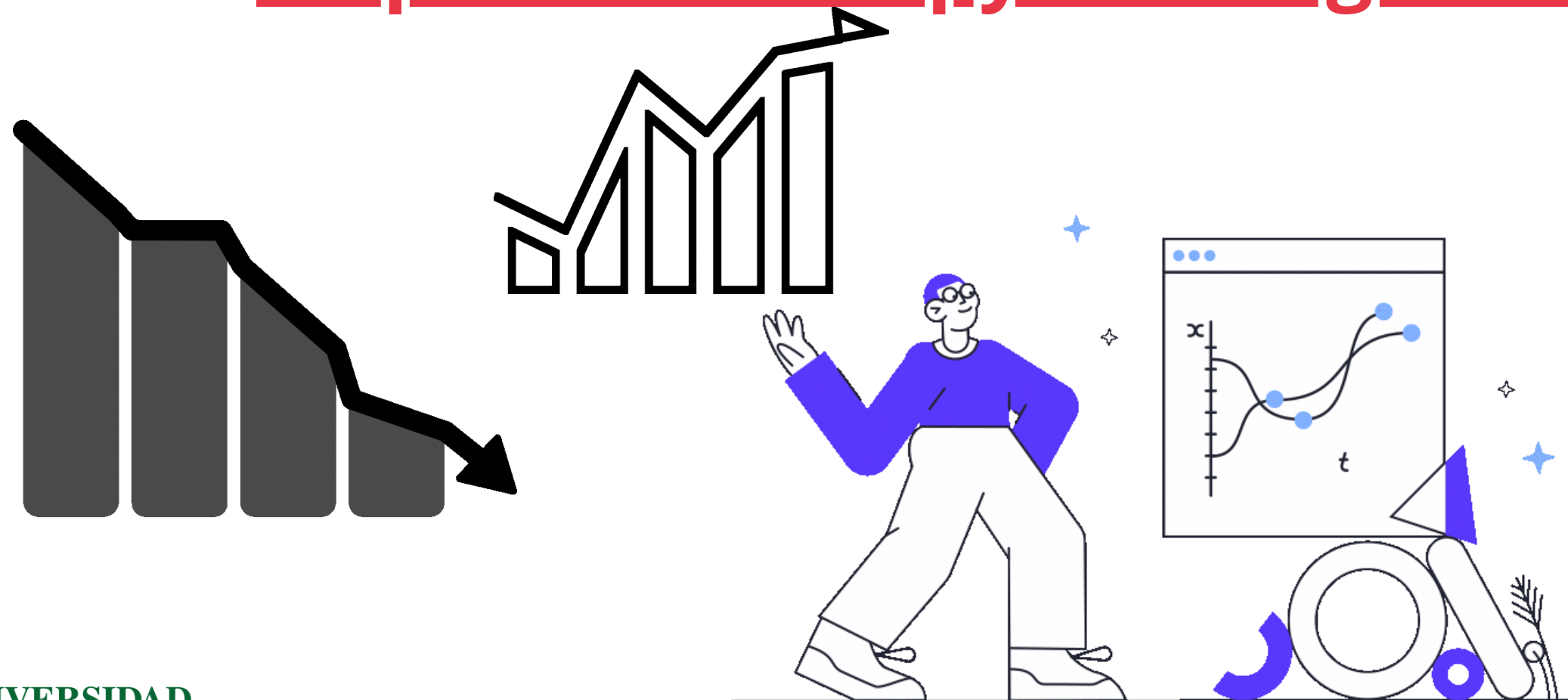
Referencia



<https://pandas.pydata.org/docs/index.html>

<https://pandas.pydata.org/Pandas Cheat Sheet.pdf>

<https://seaborn.pydata.org/tutorial/introduction>



GRACIAS